

TEORÍA (8 puntos)

Apellidos y nombre:

Pregunta 1. ¿Cuáles son las tres fases principales en el desarrollo guiado por las pruebas (*test driven development*, TDD)? ¿Qué color se suele asociar a dos de ellas?

Pregunta 2. Técnicas basadas en puntos de decisión.

Dado el siguiente código fuente escrito en un lenguaje de operadores lógicos no cortocircuitados:

```
if a > 0 and b > 0
    then r = 0;
    else r = -1;
end if;
```

y los siguientes casos de prueba posibles:

Caso	a	b	Resultado esperado
1	5	5	r = 0
2	5	-2	r = -1
3	-2	5	r = -1
4	-2	-2	r = -1

Selecciona un conjunto mínimo de casos de prueba para conseguir un 100 % de cobertura de los siguientes tipos:

- Cobertura de decisiones:
- Cobertura de condiciones:
- Cobertura de condiciones y decisiones:
- Cobertura múltiple de condiciones:
- Cobertura de condiciones y decisiones modificada:

Pregunta 3. ¿En qué consiste el «Análisis estático de código»?

Pregunta 4. Describe tres métricas que permitan informar de forma específica de la efectividad de las pruebas

Pregunta 5. Describe brevemente cómo se calculan los puntos de prueba iniciales utilizados como base para aplicar la técnica de estimación denominada “Análisis de Puntos de Prueba”.

Pregunta 6. ¿En qué consiste la base de las pruebas (*test basis*)? Enumera tres elementos que se pueden utilizar como base de las pruebas.

Pregunta 7. Describe brevemente la técnica de diseño de pruebas denominada *scenario testing* y nombra alguna herramienta que permita automatizar de forma directa las pruebas diseñadas con esta técnica.

Pregunta 8. ¿Para qué tipo de pruebas son útiles las herramientas de analítica Web? Justifica tu respuesta.

Pregunta 9. Distingue las posibles formas de ejecutar las pruebas.

Pregunta 10. Enumera tres tipos distintos de objetivos de pruebas que se pueden utilizar como punto de partida para realizar el análisis de riesgos de un producto y justifica su interés.

EJERCICIO (2 puntos)**Prueba de caminos con profundidad**

El método `replaceDigits` del siguiente fragmento de código devuelve una cadena de caracteres en las que cada aparición del dígito 3 de un parámetro n se sustituye por la cadena "Foo" y cada aparición del dígito 5, por la cadena "Bar". Esto se hace tantas veces como aparezcan los dígitos 3 y 5, y en el mismo orden de aparición. Las apariciones de otros dígitos se ignoran.

A modo de ejemplo,

- `replaceDigits(1)` devuelve la cadena vacía
- `replaceDigits(3)` devuelve "Foo"
- `replaceDigits(5)` devuelve "Bar"
- `replaceDigits(5303)` devuelve "BarFooFoo"

Este método se podría utilizar como parte de la solución de la kata *FooBarQix* que practicamos en el *coding dojo* del 6 de mayo. En concreto, resuelve el requisito de que hay que sustituir cada aparición de los dígitos 3, 5 o 7 por "Foo", "Bar" o "Qix", respectivamente, en el orden en el que aparecen los dígitos. Se ha obviado la sustitución de 7 por "Qix" para simplificar el problema.

```
public static final String FOO_STRING = "Foo";
public static final String BAR_STRING = "Bar";

public String replaceDigits(int n) {
    A String result = "";
    String digitString = Integer.toString(n);
    int i = 0;
    while (B i < digitString.length()) {
        if (C digitString.charAt(i) == '3') {
            D result += FOO_STRING;
        } else if (E digitString.charAt(i) == '5') {
            F result += BAR_STRING;
        }
        G i++;
    }
    H return result;
}
```

1. Obtén en primer lugar el grafo de flujo asociado, de acuerdo con la correspondencia entre instrucciones/condiciones y nodos que se indica en el propio código (0,25 puntos).
2. Tras haber realizado el grafo de flujo, y a la vista del código, identifica un par de aristas cuya ejecución consecutiva es imposible y explica por qué (0,25 puntos).
3. Aplica la técnica de pruebas de caminos con **profundidad 2** para obtener las situaciones de prueba y el conjunto de caminos completos correspondiente (0,75 puntos).
4. Finalmente, establece los casos de prueba correspondientes a los caminos obtenidos, incluyendo los valores de los parámetros del método y su valor devuelto (0,75 puntos).

Se recuerda que el método `Integer.toString()` devuelve una cadena de caracteres que representa el valor de su parámetro entero expresado en base 10. Por ejemplo, `Integer.toString(5303)` devuelve la cadena "5303".